

# TokenAuthenticator - How It Works

TranscilMobileApp | OkHttp auth recovery reference | July 2026

This note explains where TokenAuthenticator lives in the project, how it is wired, and exactly what happens when an API call returns HTTP 401. Use it as a quick onboarding / debugging reference.

## 1. What it is

TokenAuthenticator is an OkHttp Authenticator. OkHttp calls it automatically when a response is HTTP 401 Unauthorized. Its job is not to attach tokens on every request (that is AuthInterceptor). Its job is recovery: try to get a new access token with the refresh token, then ask OkHttp to retry the failed request once.

- Return a new Request -> OkHttp retries the call with the new Authorization header.
- Return null -> OkHttp gives up; the caller receives the 401 failure.

## 2. Where it lives in the project

Package path under app/src/main/java:

```
com.example.transcilmobileapp
+-- data/
|   +-- local/
|       |   +-- TokenStore.kt           # Encrypted access + refresh tokens
|       +-- network/
|           +-- ApiClient.kt             # Wires OkHttp + Retrofit
|           +-- AuthInterceptor.kt        # Adds Bearer on every request
|           +-- TokenAuthenticator.kt     # THIS FILE - 401 recovery
|           +-- AuthTokenRefresher.kt    # POST /v1/auth/refresh (plain OkHttp)
|           +-- TranscilApi.kt           # Retrofit API surface
+-- repository/
    +-- AuthRepository.kt                # Login / verify / logout (app-level)
```

Unit tests:

```
app/src/test/java/.../data/network/TokenAuthenticatorTest.kt
```

## 3. How it is wired

ApiClient builds one shared OkHttpClient used by Retrofit (transcilApi / demoApi). Order matters:

```
OkHttpClient.Builder()
    .addInterceptor(AuthInterceptor()) // 1) attach Bearer if present
    .addInterceptor(logging)           // 2) debug-only BODY logs
    .authenticator(TokenAuthenticator()) // 3) on 401, try refresh + retry
    .build()
```

*Important: Interceptor runs on the way out for every call. Authenticator runs only after a 401. Refresh itself uses a separate plain OkHttp client inside AuthTokenRefresher (no authenticator) so a failed refresh cannot recurse forever.*

## 4. End-to-end happy path

Normal authenticated call when the access token is still valid:

- App calls Retrofit API (e.g. get onboarding).
- AuthInterceptor reads TokenStore access token and sets Authorization: Bearer <access>.
- Server returns 200. Authenticator is never called.

When the access token is expired / rejected:

- 1. Request goes out with old Bearer (via AuthInterceptor).
- 2. Server responds 401.
- 3. OkHttp invokes TokenAuthenticator.authenticate(...).
- 4. Authenticator calls AuthTokenRefresher.refreshBlocking(refreshToken).
- 5. Refresher POSTs { refresh\_token } to /v1/auth/refresh with Idempotency-Key.

- 6. On success, `TokenStore.save(newAccess, sameRefresh)`.
- 7. Authenticator returns the original request rebuilt with Bearer <newAccess>.
- 8. OkHttp retries once. Caller usually sees success and never notices the 401.

## 5. Exact decision flow inside authenticate()

Pseudo-flow matching TokenAuthenticator.kt:

```
authenticate(response):
    if request path ends with /auth/refresh or /token/refresh:
        TokenStore.clear(); return null           # never refresh the refresh call

    if responseCount(response) >= 2:
        TokenStore.clear(); return null           # already retried once

    failedAccess = Bearer token from failed request

    synchronized(lock):                          # single-flight across threads
        currentAccess = TokenStore.getAccessToken()
        if currentAccess is non-blank AND currentAccess != failedAccess:
            return retry with currentAccess        # another thread already refreshed

        refresh = TokenStore.getRefreshToken()
        if refresh blank:
            TokenStore.clear(); return null

        newAccess = AuthTokenRefresher.refreshBlocking(refresh)
        if newAccess blank / failed:
            TokenStore.clear(); return null

        TokenStore.save(newAccess, refresh)
        return retry with newAccess
```

## 6. Why each guard exists

### A) Skip refresh endpoint

If /v1/auth/refresh itself returns 401, refreshing again would loop. The authenticator clears tokens and returns null so the session ends cleanly.

### B) responseCount >= 2

OkHttp chains priorResponse for each retry. Count of 1 = first 401. Count of 2 means we already retried once and still got 401. Stop and clear tokens.

### C) synchronized(lock) + reuse newer token

Multiple API calls can 401 at the same time. Without a lock, each would call refresh. With the lock, only one refresh runs. Waiting threads see TokenStore already has a newer access token than the one that failed, and reuse it.

### D) Clear on failure

If there is no refresh token, or refresh fails, TokenStore.clear() removes access + refresh so the app can route the user back to login (via AuthSession / sign-out flows).

## 7. Related components (roles)

- TokenStore - EncryptedSharedPreferences + in-memory cache for access/refresh.
- AuthInterceptor - Always attaches current access token before the request leaves.
- TokenAuthenticator - Only on 401; refresh + rebuild request.
- AuthTokenRefresher - Blocking POST /v1/auth/refresh on a plain OkHttp client.
- AuthRepository - App-level start/verify/refresh/logout used by UI ViewModels.
- AuthSession - Cold-start routing and sign-out glue (not part of OkHttp).

## 8. Mental model diagram

```
UI / ViewModel / Repository
|
v
Retrofit (TranscilApi)
|
v
OkHttpClient (ApiClient)
|-- AuthInterceptor -----> adds Bearer from TokenStore
|-- (optional debug log)
+-- TokenAuthenticator --> only if HTTP 401
    |
    +--> AuthTokenRefresher --> POST /v1/auth/refresh
    |
    +--> TokenStore.save(new access)
    |
    +-- retry original request once
```

## 9. Quick debug checklist

- 401 with no retry? Check TokenStore has a refresh token.
- Infinite refresh loops? Confirm refresh uses AuthTokenRefresher (no authenticator).
- Two refreshes for one screen load? Look for missing synchronized reuse path.
- User stuck logged-in after bad refresh? Confirm TokenStore.clear() on failure.
- Unit coverage: TokenAuthenticatorTest (refresh+retry, concurrent reuse, refresh-path 401).

*Files of truth: TokenAuthenticator.kt, AuthTokenRefresher.kt, AuthInterceptor.kt, ApiClient.kt, TokenStore.kt under app/src/main/java/com/example/transcilmobileapp/data/.*